

Universidad Politécnica de Madrid
Escuela Técnica Superior de Ingeniería Informática

Clasificación Automática de
Afirmaciones Políticas
mediante Procesamiento del Lenguaje
Natural

Competencia Kaggle — Asignatura ABID

Tarea: Clasificación binaria de afirmaciones políticas
Dataset: 8.950 muestras de entrenamiento + 3.860 de test
Métrica: Macro F1 Score
Plataforma: Kaggle

Grupo 1

Iván Omar Angeles Martínez

Marla González Escobar

Belén María Iniesta Tejera

Silvia Rodríguez Hernández

Rosali Rodriguez Ureña

Marilyn Salazar Martínez

Mayo de 2025

Índice

1. Introducción	2
2. Descripción del problema	2
3. Descripción del conjunto de datos	2
4. Descripción de tareas comunes	4
5. Evaluación	6
5.1. Modelos evaluados	6
5.1.1. Modelos lineales	6
5.1.2. Modelos <i>ensemble</i>	6
5.1.3. Transformers	7
5.2. Resultados	7
6. Análisis y conclusiones conjuntas	9
6.1. Análisis del preprocesamiento y la generación de variables	9
6.2. Conclusiones	10

1. Introducción

La detección automática de afirmaciones falsas es una tarea cada vez más relevante dentro del Procesamiento del Lenguaje Natural, especialmente en contextos políticos, donde la desinformación puede influir de forma significativa en la opinión pública. Este proyecto aborda un problema de clasificación binaria: predecir si una afirmación política es verdadera o falsa a partir de su contenido textual e información contextual como el emisor, su afiliación política o el tema tratado.

Para resolver el problema se evaluaron distintos enfoques de aprendizaje automático, desde modelos clásicos hasta modelos más avanzados basados en *embeddings* semánticos, técnicas de *ensemble* y *boosting*.

Esta memoria resume el análisis del conjunto de datos, el preprocesamiento realizado, los modelos desarrollados, la metodología de evaluación y las principales conclusiones obtenidas.

2. Descripción del problema

El objetivo del proyecto consiste en desarrollar un sistema capaz de clasificar afirmaciones políticas como verdaderas o falsas utilizando técnicas de Procesamiento del Lenguaje Natural y aprendizaje automático. Se trata de un problema de clasificación binaria, donde la variable objetivo toma el valor 1 para las afirmaciones verdaderas y 0 para las afirmaciones falsas.

El *input* principal es el texto de la afirmación, pero también se cuenta con metadatos como tema, *speaker*, ocupación, estado y afiliación política, que pueden aportar señal adicional al modelo.

El reto central es que la veracidad no siempre se puede inferir solo del texto. Dos afirmaciones pueden ser lingüísticamente similares y pertenecer a clases opuestas, por lo que no basta con un único enfoque. A eso se suma el *class imbalance* del *dataset*, razón por la que se usa **Macro F1** como métrica principal, ya que permite evaluar el modelo de forma balanceada entre ambas clases. Se probaron varios enfoques: desde modelos clásicos con representaciones tipo TF-IDF [1] hasta modelos más avanzados con *semantic embeddings* [2] y técnicas de *ensemble* [3, 4, 5], buscando el mejor *trade-off* entre precisión y generalización.

3. Descripción del conjunto de datos

El conjunto de datos utilizado [6] contiene afirmaciones políticas acompañadas de información contextual relacionada con cada una de ellas. Cada registro incluye el texto de la afirmación, recogido en la variable `statement`, junto con distintos metadatos como

el tema tratado (**subject**), el hablante (**speaker**), su ocupación (**speaker_job**), el estado (**state_info**) y la afiliación política (**party_affiliation**). La variable objetivo es **label**, donde el valor 1 representa una afirmación verdadera y el valor 0 una afirmación falsa.

Cuadro 1: Variables del *dataset*.

Variable	Tipo	Descripción
id	Identificador	Solo para trazabilidad; excluido del modelo.
label	Target binario	0 = verdadero, 1 = falso.
statement	Texto	El texto de la afirmación política.
subject	Categórica	Tema(s) de la afirmación; puede contener varios.
speaker	Categórica	Persona que realiza la afirmación.
speaker_job	Categórica	Cargo del hablante (27,7 % de valores ausentes).
state_info	Categórica	Estado geográfico de EE. UU.
party_affiliation	Categórica	Afiliación política del hablante.

Durante el análisis exploratorio inicial se observó que el *dataset* contiene **8.950 muestras** y presenta un cierto desbalanceo entre clases. Aproximadamente el 65 % de las muestras pertenecen a la clase verdadera, mientras que el 35 % corresponden a afirmaciones falsas. Este desequilibrio se tuvo en cuenta en la elección de la métrica de evaluación, dando prioridad a Macro F1, ya que permite valorar de forma más equilibrada el rendimiento en ambas clases.

También se detectaron valores nulos principalmente en las variables **speaker_job** y **state_info**. En lugar de eliminar estos registros, los valores faltantes se sustituyeron por la categoría **unknown**, conservando así todas las muestras disponibles y permitiendo que los modelos pudieran tratar la ausencia de información como una categoría más.

Otro aspecto relevante del *dataset* es la alta cardinalidad de algunas variables categóricas, especialmente **speaker** y **speaker_job**, donde muchos valores aparecen muy pocas veces. Para reducir ruido y mejorar la generalización, las categorías poco frecuentes se agruparon bajo la etiqueta **other**.

Por último, la columna **subject** podía contener varios temas separados por comas para una misma afirmación. Por ello, se realizó una limpieza de esta variable y se generaron características auxiliares, como una versión normalizada del texto y el número de temas asociados a cada afirmación.

4. Descripción de tareas comunes

El desarrollo del proyecto se llevó a cabo de forma colaborativa, distribuyendo las tareas entre los distintos integrantes del grupo. Cada miembro trabajó sobre diferentes etapas del *pipeline* o sobre distintos modelos, pero las decisiones principales relacionadas con el preprocesamiento, la evaluación y la comparación de resultados se realizaron de forma conjunta para mantener coherencia en el trabajo.

La planificación del trabajo se estructuró en varias fases: análisis exploratorio del *dataset*, limpieza y preprocesamiento de datos, generación de variables, entrenamiento y ajuste de modelos, evaluación de resultados y documentación final. Esta organización permitió comparar los distintos experimentos bajo criterios similares y seleccionar las estrategias con mejor rendimiento.

Para garantizar la reproducibilidad y el control de versiones, se utilizó GitHub [7] como repositorio común del proyecto. El desarrollo y la experimentación se realizaron principalmente en Jupyter Notebooks utilizando Python [8] y librerías de aprendizaje automático como `pandas` [9], `scikit-learn` [1], XGBoost [5], LightGBM [3], CatBoost [4] y Hugging Face Transformers [10]. La evaluación de resultados y generación de *submissions* se llevó a cabo mediante Kaggle [11].

Para la búsqueda de hiperparámetros se utilizaron técnicas como `RandomizedSearchCV` y `HalvingGridSearchCV` [1], permitiendo explorar distintas configuraciones de forma eficiente. En los modelos Transformer también se ajustaron parámetros relacionados con el entrenamiento profundo, como *learning rate*, *batch size*, *dropout* y número de épocas.

Finalmente, se compararon distintos enfoques de modelado, desde modelos lineales clásicos basados en TF-IDF hasta modelos *ensemble* y arquitecturas Transformer avanzadas, analizando tanto el rendimiento obtenido como la capacidad de generalización de cada uno de ellos.

Herramientas y tecnologías

Cuadro 2: Herramientas utilizadas en el proyecto.

Categoría	Herramienta	Uso
Lenguaje	Python 3.x [8]	Todo el <i>pipeline</i> .
ML clásico	scikit-learn [1]	LR, RFC, StratifiedKFold, RandomizedSearchCV, escalado, codificación.
Gradient Boosting	LightGBM [3]	Clasificador <i>gradient boosting</i> hoja a hoja.
Gradient Boosting	CatBoost [4]	Soporte nativo de categóricas y <i>ordered boosting</i> .
Gradient Boosting	XGBoost [5]	Modelo base en el <i>stacking</i> .
<i>Embeddings</i>	sentence-transformers [2]	all-MiniLM-L6-v2 (384 dim) y all-mpnet-base-v2 (768 dim).
NLP	NLTK [12]	Porter/Snowball <i>stemmer</i> , WordNet <i>lemmatizer</i> .
NER	spaCy [13]	Conteo de entidades nombradas (PERSON, ORG, GPE, DATE...).
Transformers	HuggingFace Transformers [10]	<i>Fine-tuning</i> de DeBERTa-v3-small/base [14].
<i>Fine-tuning</i> efic.	PEFT + bitsandbytes [15, 16]	LoRA sobre modelos 7B (experimento 5).
<i>Tracking</i>	Weights & Biases [17]	Métricas, curvas, importancia de <i>features</i> , artefactos.
Serialización	joblib [18]	Guardado de modelos, vectorizadores y <i>threshold</i> .
Control de versiones	GitHub [7]	Repositorio común del equipo.
Evaluación	Kaggle [11]	Generación y envío de <i>submissions</i> .

5. Evaluación

5.1. Modelos evaluados

Con el objetivo de comparar distintos enfoques para la detección de afirmaciones falsas, se evaluaron modelos de diferente complejidad, desde algoritmos lineales clásicos hasta modelos *ensemble* y arquitecturas Transformer. La comparación permitió analizar tanto el rendimiento predictivo como la capacidad de generalización de cada técnica sobre el *dataset*.

Los modelos fueron entrenados utilizando la misma estrategia de validación y conjuntos de datos procesados, adaptando únicamente la representación de las variables según las necesidades de cada algoritmo. Las métricas principales empleadas para la comparación fueron **Macro F1** y **ROC-AUC**, ya que el *dataset* presentaba desbalanceo entre clases.

5.1.1. Modelos lineales

Los primeros modelos evaluados fueron algoritmos lineales clásicos basados en representaciones TF-IDF del texto [1]. Se utilizaron principalmente Logistic Regression y Linear SVM, ya que suelen ofrecer buenos resultados como *baseline* en problemas de clasificación textual.

Para estos modelos se emplearon representaciones TF-IDF con n -gramas de palabras y caracteres, además de variables categóricas relacionadas con el contexto político y el hablante. También se probaron distintas configuraciones de regularización y balanceo de clases mediante `class_weight="balanced"`.

La Regresión Logística destacó por su simplicidad, velocidad de entrenamiento e interpretabilidad, obteniendo resultados competitivos en las primeras fases del proyecto. Por otro lado, Linear SVM permitió mejorar ligeramente la separación entre clases, especialmente al incorporar n -gramas de caracteres y calibración de probabilidades.

Aunque estos modelos ofrecieron *baselines* sólidos, presentaban limitaciones para capturar relaciones semánticas complejas entre el contenido textual y la información contextual.

5.1.2. Modelos *ensemble*

Posteriormente se evaluaron distintos modelos *ensemble* y basados en árboles de decisión, incluyendo Random Forest [1], XGBoost [5], LightGBM [3] y CatBoost [4]. Estos modelos permitieron capturar relaciones no lineales entre las variables y aprovechar mejor las interacciones entre texto y *metadata* contextual.

En Random Forest se utilizaron *embeddings* semánticos [2] y variables derivadas relacionadas con el historial del hablante y el contexto político. Este modelo mostró

una mejora significativa respecto a los modelos lineales, especialmente gracias al uso de *embeddings* y variables agregadas.

XGBoost [5] y LightGBM [3] se emplearon junto con selección de características y ajuste de hiperparámetros mediante búsqueda aleatoria. Estos modelos consiguieron mejorar el rendimiento en ROC-AUC, aunque también mostraron sensibilidad al sobreajuste en determinadas variables relacionadas con el historial del hablante.

CatBoost [4] obtuvo uno de los mejores rendimientos entre los modelos individuales, gracias a su capacidad para manejar variables categóricas de forma nativa y reducir el impacto del *leakage* mediante *ordered boosting*.

Finalmente, se implementó una estrategia de *stacking* combinando varios modelos base mediante un meta-clasificador Logistic Regression [1]. Este enfoque permitió aprovechar la complementariedad entre modelos y mejorar tanto Macro F1 como ROC-AUC respecto a los modelos individuales.

5.1.3. Transformers

La última familia de modelos evaluados estuvo basada en arquitecturas Transformer preentrenadas [10], especialmente DeBERTa [14]. Estos modelos permiten capturar relaciones contextuales y semánticas más complejas que los enfoques basados únicamente en TF-IDF o *embeddings* estáticos.

Para los Transformers se utilizó texto enriquecido con *metadata* contextual, incorporando información como el hablante, el tema y la afiliación política directamente en la entrada textual del modelo. Además, se ajustaron hiperparámetros como *learning rate*, *batch size*, *dropout* y número de épocas para optimizar el rendimiento. Para los experimentos de *fine-tuning* eficiente se emplearon las librerías PEFT y bitsandbytes [15, 16].

Los modelos Transformer mostraron mejores resultados que los modelos clásicos cuando se combinaron con información contextual. Sin embargo, el mejor rendimiento global se obtuvo mediante estrategias híbridas de *late fusion*, donde las predicciones de DeBERTa [14] se integraban dentro del *stacking* junto con modelos *ensemble* tradicionales.

El mejor modelo del proyecto fue una combinación de DeBERTa-v3-base y *stacking*, alcanzando los valores más altos tanto en Macro F1 como en ROC-AUC.

5.2. Resultados

La evaluación de los modelos se realizó utilizando validación cruzada estratificada de cinco *folds* y un conjunto *holdout* reservado para la evaluación final [1]. Debido al desbalanceo presente en el *dataset*, las métricas principales utilizadas fueron Macro F1 y ROC-AUC, ya que permiten evaluar de forma más equilibrada el rendimiento sobre ambas clases.

A lo largo del proyecto se observó una mejora progresiva del rendimiento al incorporar representaciones textuales más avanzadas, variables contextuales y técnicas de combinación de modelos. Los modelos lineales basados en TF-IDF proporcionaron *base-lines* sólidos, mientras que los modelos *ensemble* y Transformer consiguieron capturar relaciones más complejas entre el texto y la *metadata* política.

Los resultados finales obtenidos por los modelos más relevantes se muestran en la Tabla 3.

Cuadro 3: Resultados en *holdout* de los modelos evaluados (ordenados por Macro F1).

Modelo	Macro F1	ROC-AUC	Umbral
LR <i>baseline</i> (TF-IDF)	0,604	0,654	0,50
LR + <i>nested CV</i> + <i>embeddings</i>	0,611	0,660	—
Random Forest (embeddings + NER + <code>job_true_rate</code>)	0,621	0,667	0,58
LGBM Opción B (sin <code>speaker_true_rate</code>)	0,618	0,679	0,52
CatBoost + Opción B	0,629	0,674	0,46
<i>Stacking</i> (<code>all-mpnet-base-v2</code> , 768 dim)	0,643	0,700	0,62
<i>Late Fusion</i> + DeBERTa-v3-small	0,644	0,709	—
<i>Late Fusion</i> + DeBERTa-v3-base	0,647	0,711	—

Uno de los factores que más contribuyó a la mejora del rendimiento fue la incorporación de *embeddings* semánticos [2] y variables contextuales relacionadas con el hablante y el tema de la afirmación. También se comprobó que algunas variables históricas, aunque muy predictivas, podían introducir problemas de sobreajuste si no se calculaban correctamente mediante estrategias *out-of-fold*.

Los modelos Transformer [10, 14] mejoraron notablemente al incorporar *metadata* contextual directamente en el texto de entrada. Sin embargo, los mejores resultados no se obtuvieron con modelos *standalone*, sino mediante estrategias híbridas de *late fusion* que combinaban las fortalezas de los Transformers y los modelos *ensemble* tradicionales [3, 4, 5].

En general, los resultados muestran que la combinación de distintas representaciones textuales y enfoques de modelado permitió obtener un sistema más robusto y con mejor capacidad de generalización que cualquiera de los modelos individuales por separado.

6. Análisis y conclusiones conjuntas

6.1. Análisis del preprocesamiento y la generación de variables

El preprocesamiento de datos y la generación de variables tuvieron un papel fundamental en el rendimiento de los modelos desarrollados. El objetivo principal fue reducir el ruido del *dataset*, representar correctamente la información textual y contextual, y evitar problemas de sobreajuste o *leakage* durante el entrenamiento.

En primer lugar, se realizó una limpieza general de las variables categóricas y textuales. Las columnas de texto se normalizaron convirtiendo todo el contenido a minúsculas, eliminando espacios innecesarios y estandarizando formatos inconsistentes. Los valores nulos de variables como `speaker_job` y `state_info` se reemplazaron por categorías específicas como `unknown` o `missing`, evitando la pérdida de información asociada a la ausencia de datos.

Para los modelos clásicos basados en TF-IDF [1] se aplicó un preprocesamiento textual más agresivo sobre la columna `statement`. Este proceso incluyó eliminación de puntuación, normalización de contracciones, reemplazo de números por un token genérico, tokenización, eliminación de *stopwords* y lematización con NLTK [12]. Además, se conservaron negaciones como *not* o *never*, ya que aportaban información relevante para la clasificación.

La representación textual se realizó principalmente mediante TF-IDF con *n*-gramas de palabras y caracteres [1]. También se utilizaron *embeddings* semánticos generados con Sentence Transformers [2], como `all-MiniLM-L6-v2` y `all-mpnet-base-v2`, permitiendo capturar relaciones semánticas más complejas entre afirmaciones.

En las variables categóricas se aplicaron técnicas de reducción de cardinalidad agrupando categorías poco frecuentes bajo la etiqueta `other`. Esto se utilizó especialmente en variables como `speaker`, `speaker_job` y `subject`, donde existía una gran cantidad de valores con muy pocas apariciones.

La variable `subject` requería un tratamiento especial debido a que una misma afirmación podía pertenecer a varios temas simultáneamente. Para ello se utilizó `MultiLabelBinarizer` [1], generando variables binarias independientes para cada tema relevante.

Además, se generaron variables derivadas relacionadas con el contexto político y el comportamiento histórico de los hablantes. Entre ellas destacan las tasas históricas de veracidad asociadas al hablante, partido político o tema tratado. Para evitar *leakage*, estas estadísticas se calcularon utilizando estrategias *out-of-fold* dentro de la validación cruzada [1].

Finalmente, en los modelos Transformer [10] no se aplicó una limpieza textual agresiva, ya que estos modelos aprovechan mejor el lenguaje natural original. En su lugar, se construyó un texto enriquecido que combinaba la afirmación con información

contextual como el hablante, el tema y la afiliación política, permitiendo incorporar *metadata* directamente en la entrada del modelo.

6.2. Conclusiones

En este proyecto se abordó un problema de clasificación de afirmaciones políticas [6] mediante técnicas de procesamiento del lenguaje natural y aprendizaje automático. A lo largo del desarrollo se evaluaron distintos enfoques, desde modelos lineales clásicos hasta arquitecturas Transformer [10, 14] y estrategias *ensemble* avanzadas [3, 4, 5].

Los resultados obtenidos muestran que la información contextual asociada a las afirmaciones, como el hablante, el tema o la afiliación política, tuvo un impacto muy relevante en el rendimiento de los modelos. La combinación de esta *metadata* con representaciones textuales avanzadas [2] permitió mejorar significativamente las métricas de clasificación.

Los modelos lineales basados en TF-IDF [1] ofrecieron *baselines* sólidos y eficientes, mientras que los modelos *ensemble* [3, 4, 5] lograron capturar relaciones no lineales entre variables y mejorar la capacidad predictiva. Por su parte, los modelos Transformer [10, 14] consiguieron representar mejor el contexto semántico del lenguaje natural, especialmente al incorporar información contextual directamente en el texto de entrada.

También se observó la importancia de controlar el sobreajuste y evitar *leakage* en variables relacionadas con el historial de veracidad de los hablantes. El uso de validación cruzada estratificada [1] y estrategias *out-of-fold* permitió reducir este problema y obtener estimaciones más realistas del rendimiento.

El mejor resultado del proyecto se alcanzó mediante una estrategia híbrida de *late fusion* combinando DeBERTa-v3-base [14] con *stacking* de modelos *ensemble*, obteniendo los valores más altos de Macro F1 y ROC-AUC. Esto demuestra que la combinación de modelos con mecanismos de aprendizaje diferentes puede aprovechar mejor las fortalezas de cada enfoque y mejorar la capacidad de generalización final del sistema.

En conclusión, el proyecto permitió comparar distintas técnicas de clasificación textual y comprobar cómo la integración de información contextual y estrategias híbridas puede mejorar significativamente el rendimiento en tareas de detección de desinformación política.

Referencias

- [1] scikit-learn developers. scikit-learn: Machine learning in Python — user guide, 2024. URL https://scikit-learn.org/stable/user_guide.html. Accedido: mayo 2025.

- [2] Nils Reimers and Iryna Gurevych. Sentence-transformers documentation, 2024. URL <https://www.sbert.net/>. Accedido: mayo 2025.
- [3] Microsoft / LightGBM Contributors. LightGBM documentation, 2024. URL <https://lightgbm.readthedocs.io/en/stable/>. Accedido: mayo 2025.
- [4] Yandex / CatBoost Contributors. CatBoost documentation, 2024. URL <https://catboost.ai/docs/>. Accedido: mayo 2025.
- [5] XGBoost Contributors. XGBoost documentation, 2024. URL <https://xgboost.readthedocs.io/en/stable/>. Accedido: mayo 2025.
- [6] William Yang Wang. “Liar, Liar Pants on Fire”: A new benchmark dataset for fake news detection. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL 2017)*, 2017. URL <https://arxiv.org/abs/1705.00648>.
- [7] GitHub, Inc. GitHub — where the world builds software, 2025. URL <https://github.com/>. Accedido: mayo 2025.
- [8] Python Software Foundation. Python 3 reference manual, 2024. URL <https://docs.python.org/3/>. Accedido: mayo 2025.
- [9] The pandas development team. pandas: powerful python data analysis toolkit, 2024. URL <https://pandas.pydata.org/docs/>. Accedido: mayo 2025.
- [10] Hugging Face. Transformers documentation, 2024. URL <https://huggingface.co/docs/transformers/>. Accedido: mayo 2025.
- [11] Kaggle. Kaggle — your machine learning and data science community, 2025. URL <https://www.kaggle.com/>. Accedido: mayo 2025.
- [12] NLTK Project. NLTK: Natural language toolkit documentation, 2024. URL <https://www.nltk.org/>. Accedido: mayo 2025.
- [13] Explosion AI. spaCy — industrial-strength natural language processing, 2024. URL <https://spacy.io/>. Accedido: mayo 2025.
- [14] Microsoft. DeBERTa-v3-base — hugging face model card, 2023. URL <https://huggingface.co/microsoft/deberta-v3-base>. Accedido: mayo 2025.
- [15] Hugging Face. PEFT: State-of-the-art parameter-efficient fine-tuning methods, 2024. URL <https://huggingface.co/docs/peft/>. Accedido: mayo 2025.
- [16] Hugging Face. bitsandbytes documentation, 2024. URL <https://huggingface.co/docs/bitsandbytes/>. Accedido: mayo 2025.

- [17] Weights & Biases. Weights & Biases documentation, 2024. URL <https://docs.wandb.ai/>. Accedido: mayo 2025.
- [18] joblib Contributors. joblib: Running python functions as pipeline jobs, 2024. URL <https://joblib.readthedocs.io/>. Accedido: mayo 2025.